



Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Array : linear search - $O(n)$
binary search - $O(\log n)$

Linked List : search - $O(n)$

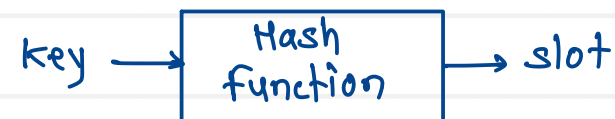
Hashing :
technique which allows to search
data in constant time ($O(1)$)

- implementation of hashing is
called as hash table / hash map

- hashing is a technique in which data can be inserted, deleted and searched in constant average time $O(1)$
- Implementation of hashing is known as hash table
- Hash table is array of fixed size in which elements are stored in key - value pairs

Array - Hash table
Index - Slot

- In hash table only unique keys are stored
- Every key is mapped with one slot of the table and this is done with the help of mathematical function known as hash function



1. Division method — $k \% m$
2. Multiplication method — $\lfloor m * (k * A) \rfloor$ $0 > A < 1$
3. Mid-square method — k^2 & find middle digit
4. Folding method — divide in two part, sum them, $\% m$
5. Cryptographic hashing — MD5, SHA-1 and SHA-256
6. Universal hashing — $((a * k + b) \% p) \% m$
7. Perfect hashing — collision free hash function for static set of key.
 - guarantees that no two keys will hash to same value

Size = 10

key	value		
8-V1		10, V3	0
3-V2			1
10-V3			2
4-V4		3, V2	3
6-V5		4, V4	4
13-V6			5
		6, V5	6
			7
		8, V1	8
			9

Hash Table

$h(k) = k \% \text{size}$ ← hash function

$h(8) = 8 \% 10 = 8$
 $h(3) = 3 \% 10 = 3$
 $h(10) = 10 \% 10 = 0$
 $h(4) = 4 \% 10 = 4$
 $h(6) = 6 \% 10 = 6$
 $h(13) = 13 \% 10 = 3$

Add: $O(1)$

1. find slot = $h(\text{key})$
2. $\text{arr}[\text{slot}] = (\text{key}, \text{value})$

Search: $O(1)$

1. find slot = $h(\text{key})$
2. return $\text{arr}[\text{slot}].\text{value}$

Delete: $O(1)$

1. find slot = $h(\text{key})$
2. $\text{arr}[\text{slot}] = \text{null}$

Collision:

- when two distinct keys yield (give) same slot of table

Collision handling/resolution techniques:

1. open addressing
 - i. linear probing
 - ii. quadratic probing
 - iii. double hashing
2. Closed addressing / chaining

} **probing:**
process of finding next
empty slot where data
can be inserted when
collision will occur

Open addressing - Linear probing

Size = 10

8-V1

3-V2

10-V3

4-V4

6-V5

13-V6

10, V3	0
	1
	2
3, V2	3
4, V4	4
13, V6	5
6, V5	6
	7
8, V1	8
	9

Hash Table

$$h(k) = k \% \text{ size}$$

$$h(k, i) = [h(k) + f(i)] \% \text{ size}$$

$$f(i) = i$$

probe number

where $i = 1, 2, 3, \dots$

$$h(13) = 13 \% 10 = 3 \text{ (c)}$$

$$1^{\text{st}} \text{ probe : } h(13, 1) = [3 + 1] \% 10 = 4 \text{ (c)}$$

$$2^{\text{nd}} \text{ probe : } h(13, 2) = [3 + 2] \% 10 = 5$$

Primary clustering:

- table becomes crowded near key position
- need to take long run of filled slots to find empty slot "near" key position

Open addressing - Quadratic probing

size = 10

	10, V3	0
8-V1		1
3-V2		2
10-V3	3, V2	3
4-V4	4, V4	4
6-V5		5
13-V6	6, V5	6
	13, V6	7
	8, V1	8
		9

Hash Table

$$h(k) = k \% \text{size}$$

$$h(k, i) = [h(k) + f(i)] \% \text{size}$$

$$f(i) = i^2$$

where $i = 1, 2, 3, \dots$

$$h(13) = 13 \% 10 = 3 \text{ (c)}$$

$$1^{\text{st}}: h(13, 1) = [3 + 1] \% 10 = 4 \text{ (c)}$$

$$2^{\text{nd}}: h(13, 2) = [3 + 4] \% 10 = 7$$

secondary clustering:

- need to take long run of filled slots to find empty slot "away" key position

- there is no guarantee of getting free slot for key value pair.

Open addressing - Quadratic probing

size = 10

	10, V3	0
8-V1		1
3-V2	23, V7	2
10-V3	3, V2	3
4-V4	4, V4	4
6-V5		5
13-V6	6, V5	6
23, V7	13, V6	7
33, V8	8, V1	8
	33, V8	9

Hash Table

$$h(k) = k \% \text{size}$$

$$h(k, i) = [h(k) + f(i)] \% \text{size}$$

$$f(i) = i^2$$

where $i = 1, 2, 3, \dots$

$$h(23) = 23 \% 10 = 3 \text{ (C)}$$

$$1^{\text{st}} : h(23, 1) = [3 + 1] \% 10 = 4 \text{ (C)}$$

$$2^{\text{nd}} : h(23, 2) = [3 + 4] \% 10 = 7 \text{ (C)}$$

$$3^{\text{rd}} : h(23, 3) = [3 + 9] \% 10 = 2$$

$$h(33) = 33 \% 10 = 3 \text{ (C)}$$

$$1^{\text{st}} : h(33, 1) = [3 + 1] \% 10 = 4 \text{ (C)}$$

$$2^{\text{nd}} : h(33, 2) = [3 + 4] \% 10 = 7 \text{ (C)}$$

$$3^{\text{rd}} : h(33, 3) = [3 + 9] \% 10 = 2 \text{ (C)}$$

$$4^{\text{th}} : h(33, 4) = [3 + 16] \% 10 = 9$$

Open addressing - Double hashing

size=11

8-V1

3-V2

10-V3

25-V4

36-V5

	0
	1
	2
3, V2	3
	4
	5
25, V4	6
	7
8, V1	8
36, V5	9
10, V3	10

Hash Table

$$h1(k) = k \% \text{size}$$

$$h2(k) = 7 - (k \% 7)$$

$$h(k, i) = [h1(k) + i * h2(k)] \% \text{size}$$

$$h_1(8) = 8 \% 11 = 8$$

$$h_1(3) = 3 \% 11 = 3$$

$$h_1(10) = 10 \% 11 = 10$$

$$h_1(25) = 25 \% 11 = 3 \text{ (C)}$$

$$h_2(25) = 7 - (25 \% 7) = 3$$

$$h(25, 1) = [3 + 1 * 3] \% 11 = 6$$

$$h_1(36) = 36 \% 11 = 3 \text{ (C)}$$

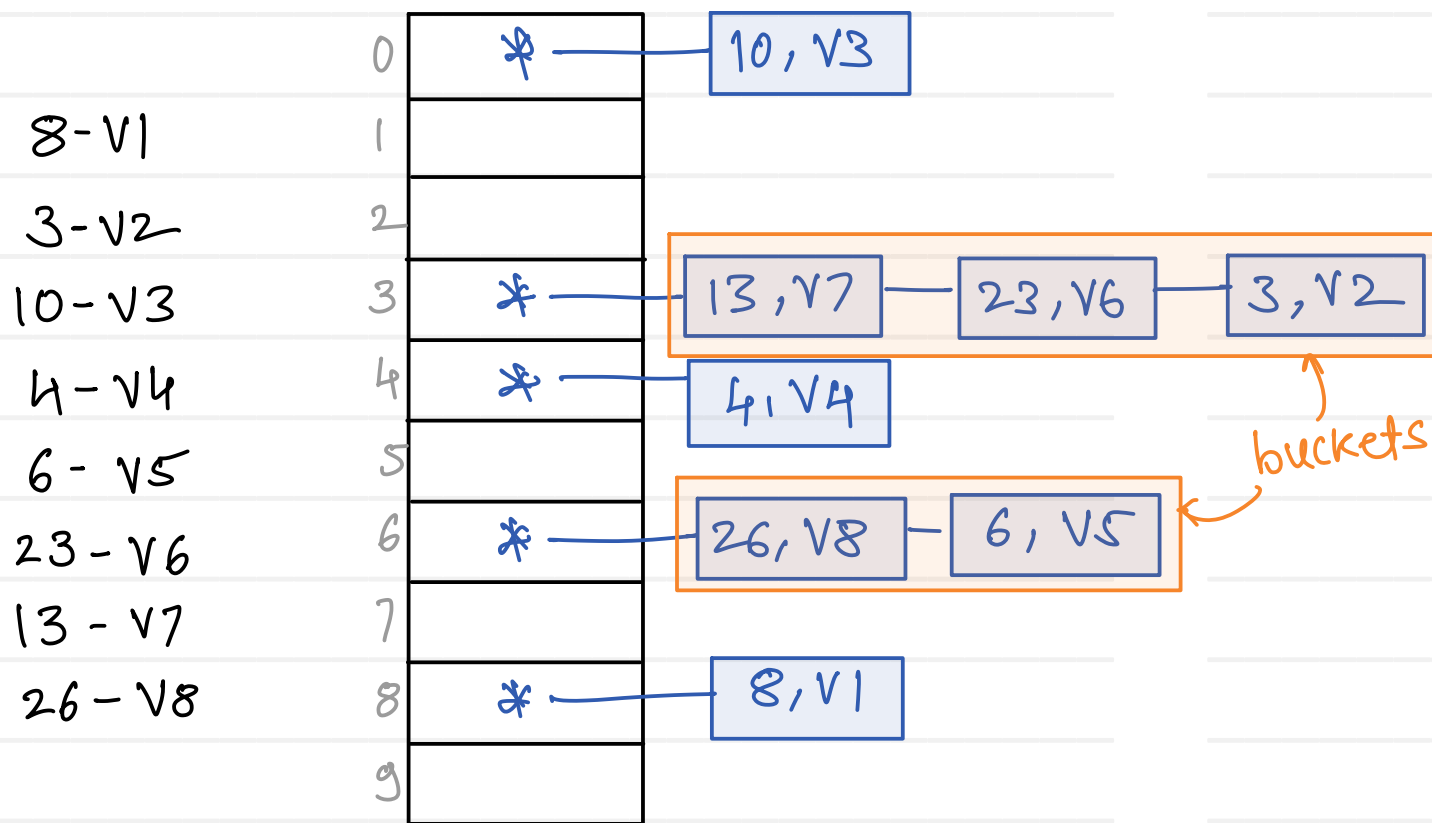
$$h_2(36) = 7 - (36 \% 7) = 6$$

$$h(36, 1) = [3 + 1 * 6] \% 11 = 9$$

Closed Addressing / Chaining / Separate Chaining

Size = 10

$$h(k) = k \% \text{ size}$$



Hash Table

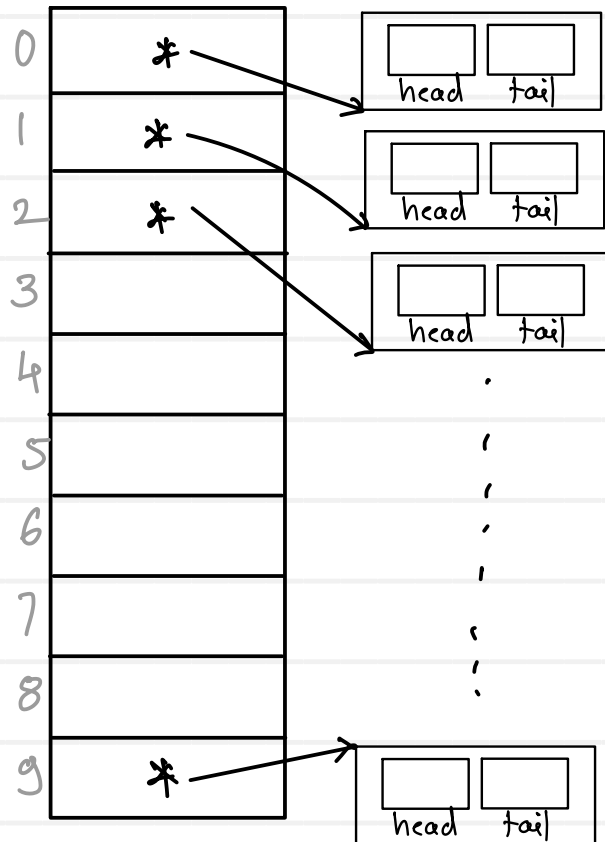
- All buckets other than key position are closed for key-value pair, that why it is called as closed addressing.

Advantage :

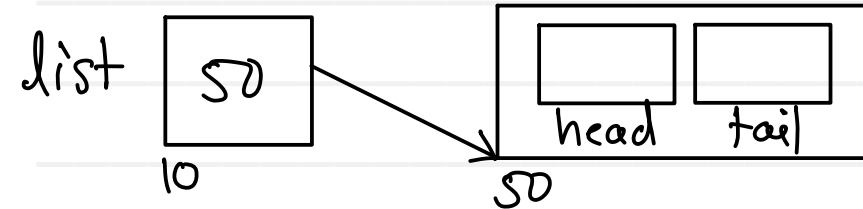
no restriction on number of key value pairs.

Disadvantages :

1. key-value pair are stored outside the table
2. space requirement of this technique is more
3. Worst case time complexity of operations is $O(n)$
↳ if multiple keys yield same slot



```
LinkedList<Integer> list = new LinkedList<>();
```



```
LinkedList<Integer> arr[] = new LinkedList<>[SIZE];
```

or

```
List<Integer> arr[] = new List[SIZE];
```

```
for (i=0; i<SIZE; i++)  
    arr[i] = new LinkedList<>();
```

Rehashing

$$\text{Load factor} = \frac{n}{N}$$

n - number of elements (key-value) present in hash table

N - number of total slots in hash table

$$n = 6, N = 10$$

$$\lambda = \frac{n}{N} = \frac{6}{10} = 0.6$$

indicates table
is 60% full

- Load factor ranges from 0 to 1.
- If $n < N$ Load factor < 1 - free slots are available
- If $n = N$ Load factor $= 1$ - free slots are not available

- In rehashing, whenever hash table will be filled more than 60 or 70 % size of hash table is increased by twice
- Existing key value pairs are remapped according to new size

Two sum

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: nums = [2,7,11,15], target = 9

Output: [0,1] ↑

HashMap

key	value

Example 2:

Input: nums = [3,2,4], target = 6

Output: [1,2] ↑

HashMap

key	value

Example 3:

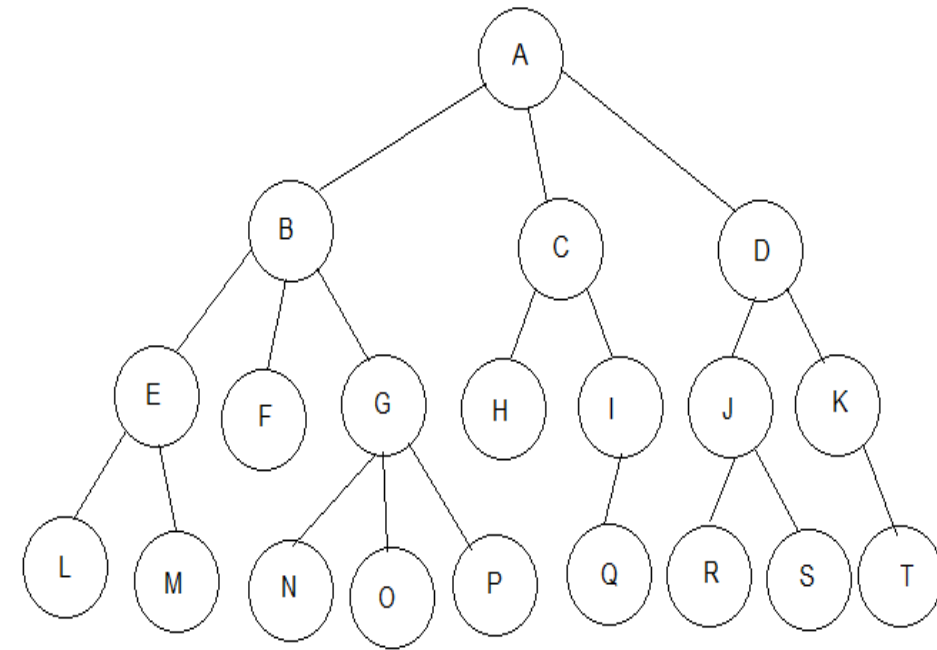
Input: nums = [3,3], target = 6

Output: [0,1]

```
int[] twoSum(int[] nums, int target) {  
    Map<Integer, Integer> tbl = new HashMap<>();  
    for (int i = 0; i < nums.length; i++) {  
        if (tbl.containsKey(target - nums[i]))  
            return new int[] {tbl.get(target - nums[i]), i};  
        tbl.put(nums[i], i);  
    }  
    return new int[] { };  
}
```

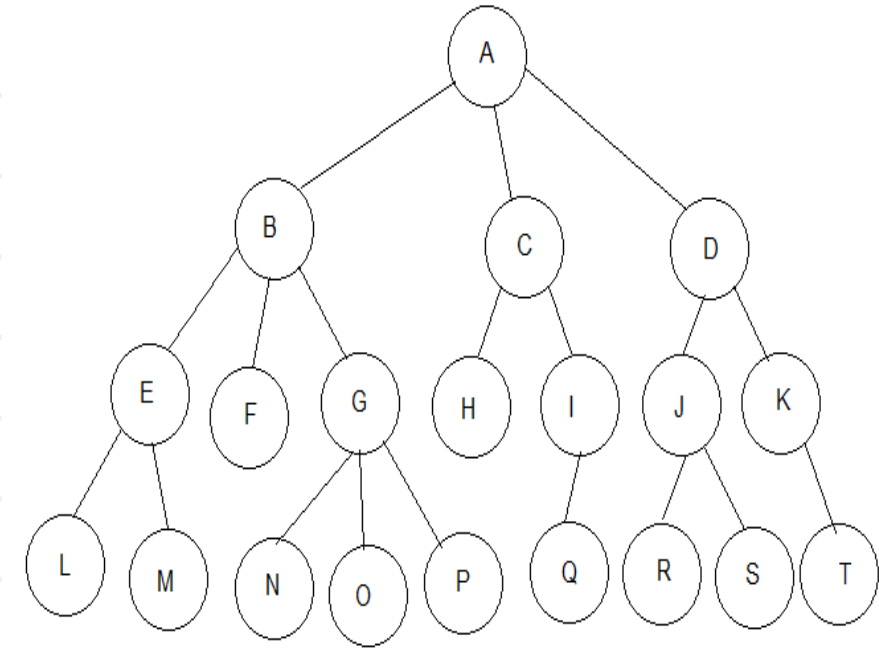
Tree - Terminologies

- **Tree** is a non linear data structure which is a finite set of nodes with one specially designated node is called as **“root”** and remaining nodes are partitioned into m disjoint subsets where each of subset is a tree..
- **Root** is a **starting point** of the tree.
- All nodes are connected in Hierarchical manner (multiple levels).
- **Parent node:-** having other child nodes connected
- **Child node:-** immediate descendant of a node
- **Leaf node:-**
 - Terminal node of the tree.
 - Leaf node does not have child nodes.
- **Ancestors:-** all nodes in the path from root to that node.
- **Descendants:-** all nodes accessible from the given node
- **Siblings:-** child nodes of the same parent



Tree - Terminologies

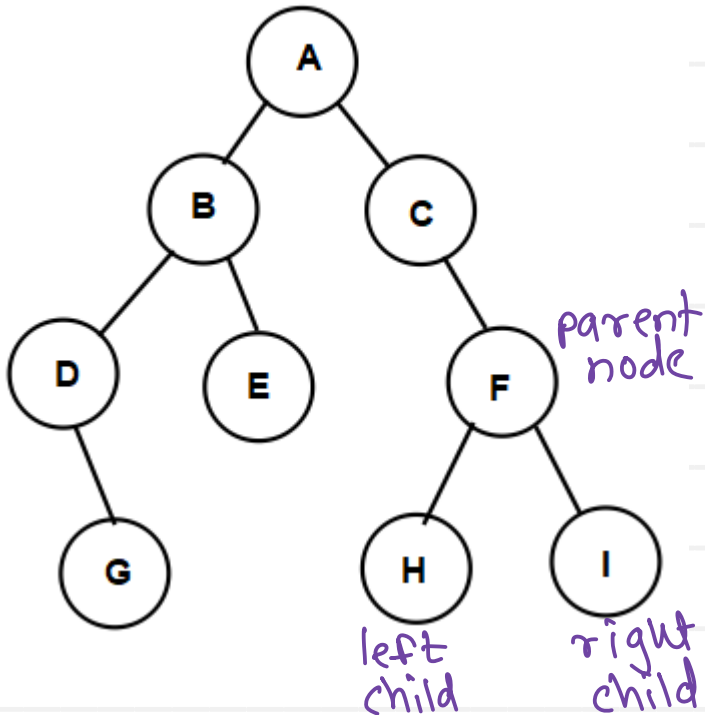
- **Degree of a node** :- number of child nodes for any given node.
- **Degree of a tree** :- Maximum degree of any node in tree.
- **Level of a node** :- indicates position of the node in tree hierarchy
 - Level of child = Level of parent + 1
 - Level of root = 0
- **Height of node** :- number of links from node to longest leaf.
- **Depth of node** :- number of links from root to that node
- **Height of a tree** :- Maximum height of a node
- **Depth of a tree** :- Maximum depth of a node
- Tree with zero nodes (ie empty tree) is called as “**Null tree**”. Height of Null tree is -1.
 - Tree can grow up to any level and any node can have any number of Childs.
 - That's why operations on tree becomes un efficient.
 - Restrictions can be applied on it to achieve efficiency and hence there are different types of trees.



Tree - Terminologies

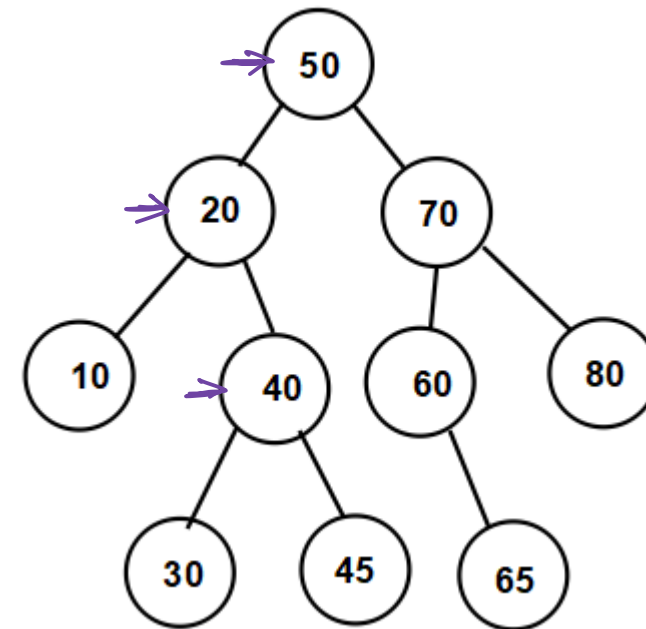
• Binary Tree

- Tree in which each node has maximum two child nodes
- Binary tree has degree 2. Hence it is also called as 2- tree



• Binary Search Tree

- Binary tree in which left child node is always smaller and right child node is always greater or equal to the parent node.
- Searching is faster
- Time complexity : $O(h)$ h – height of tree



Binary Search Tree - Implementation

Node :

data - actual data

left - reference of left child

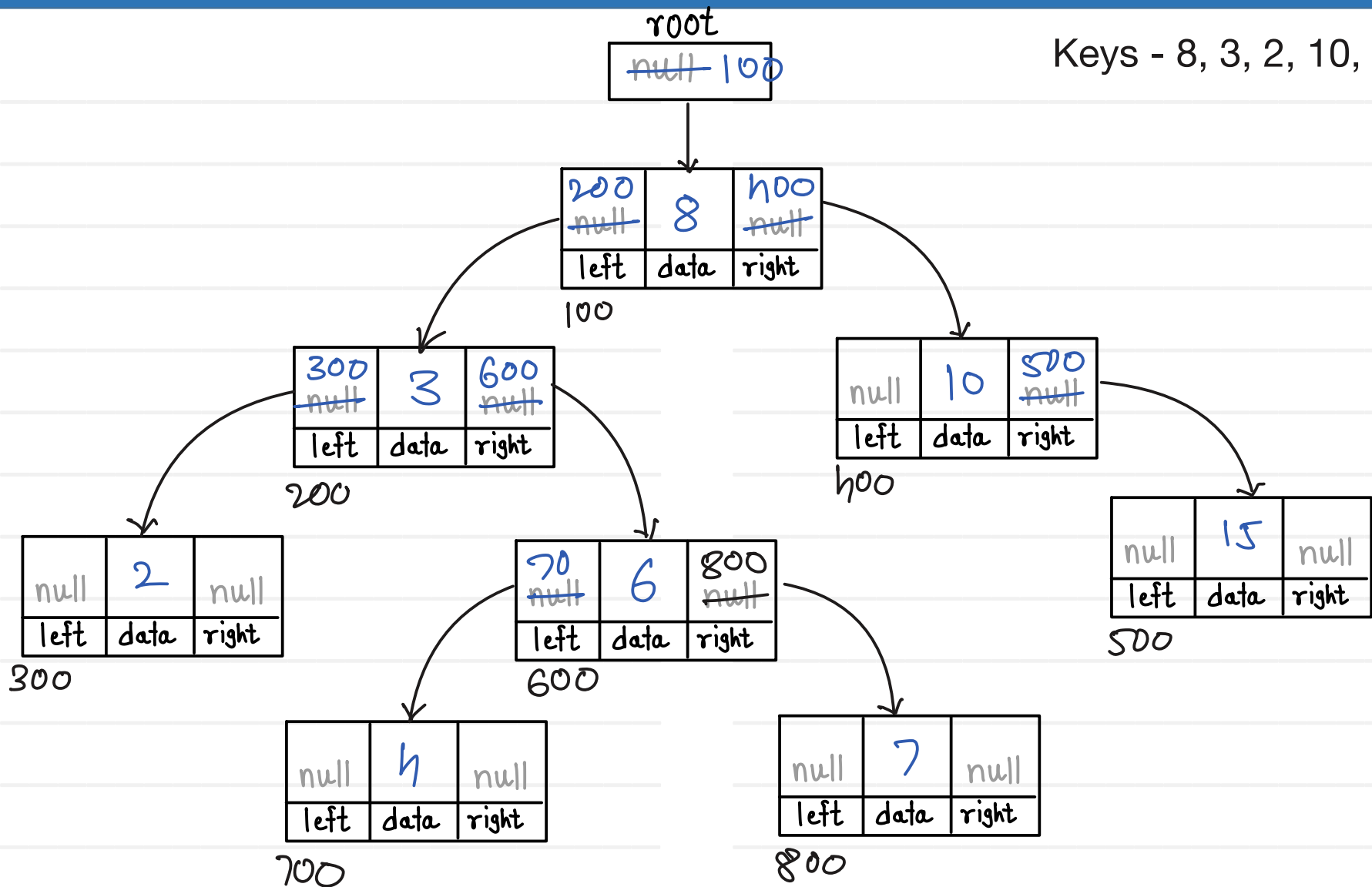
right - reference of right child

```
class Node {  
    int data;  
    Node left;  
    Node right;  
}
```

```
class BST {  
    static class Node {  
        int data;  
        Node left;  
        Node right;  
    }  
    Node root;  
    public BST() { ... }  
    public addNode(value) { ... }  
    public deleteNode(value) { ... }  
    public searchNode(key) { ... }  
    public traverse() { ... }  
    public deleteAll() { ... }  
}
```

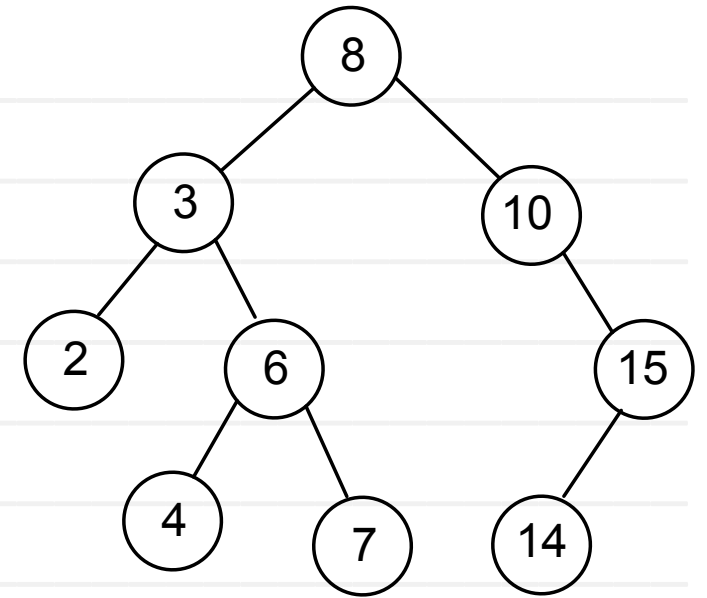
Binary Search Tree - Add Node

Keys - 8, 3, 2, 10, 15, 6, 4, 7, 14



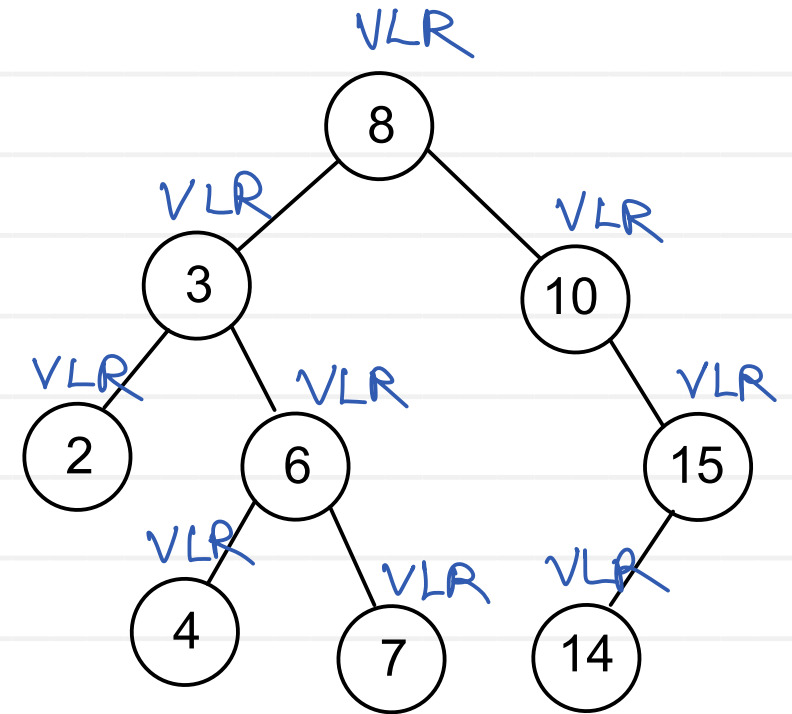
Binary Search Tree – Add Node

```
//1. create node for given value
//2. if BSTree is empty
    // add newnode into root itself
//3. if BSTree is not empty
    //3.1 create trav reference and start at root node
    //3.2 if value is less than current node data (trav.data)
        //3.2.1 if left of current node is empty
            // add newnode into left of current node
        //3.2.2 if left of current node is not empty
            // go into left of current node
    //3.3 if value is greater or equal than current node data (trav.data)
        //3.3.1 if right of current node is empty
            // add newnode into right of current node
        //3.3.2 if right of current node is not empty
            // go into right of current node
    //3.4 repeat step 3.2 and 3.3 till node is not getting added into BSTree
```



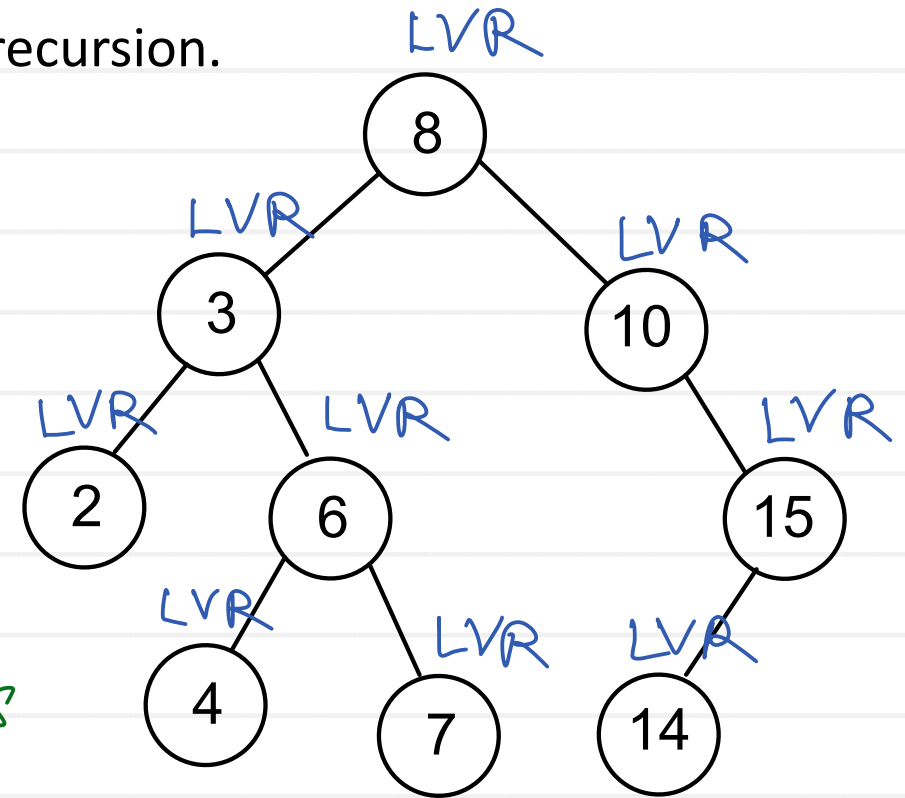
- **Pre-Order:-** V L R
- **In-order:-** L V R
- **Post-Order:-** L R V
- The traversal algorithms can be implemented easily using recursion.
- Non-recursive algorithms for implementing traversal needs stack to store node pointers.

- **Pre-Order:-**  8, 3, 2, 6, 4, 7, 10, 15, 14



- **Pre-Order:-** V L R
- **In-order:-** L V R
- **Post-Order:-** L R V
- The traversal algorithms can be implemented easily using recursion.
- Non-recursive algorithms for implementing traversal needs stack to store node pointers.

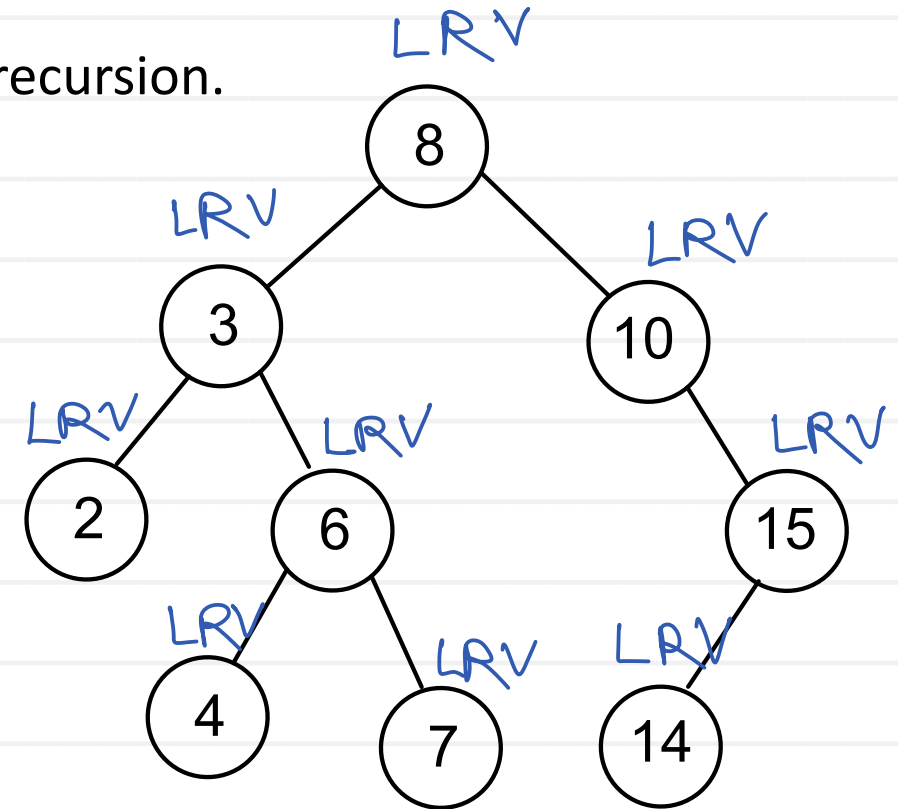
- **In-Order** :- 2, 3, 4, 6, 7, 8, 10, 14, 15
for BST, inorder traversal is always sorted.

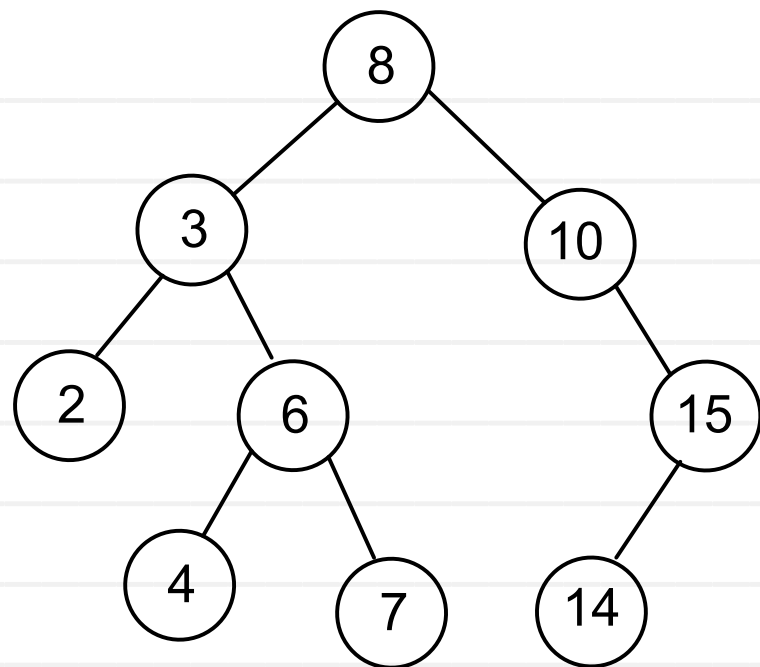


- **Pre-Order:-** V L R
- **In-order:-** L V R
- **Post-Order:-** L R V
- The traversal algorithms can be implemented easily using recursion.
- Non-recursive algorithms for implementing traversal needs stack to store node pointers.

• **Post-Order** :- 2, 4, 7, 6, 3, 14, 15, 10, 8

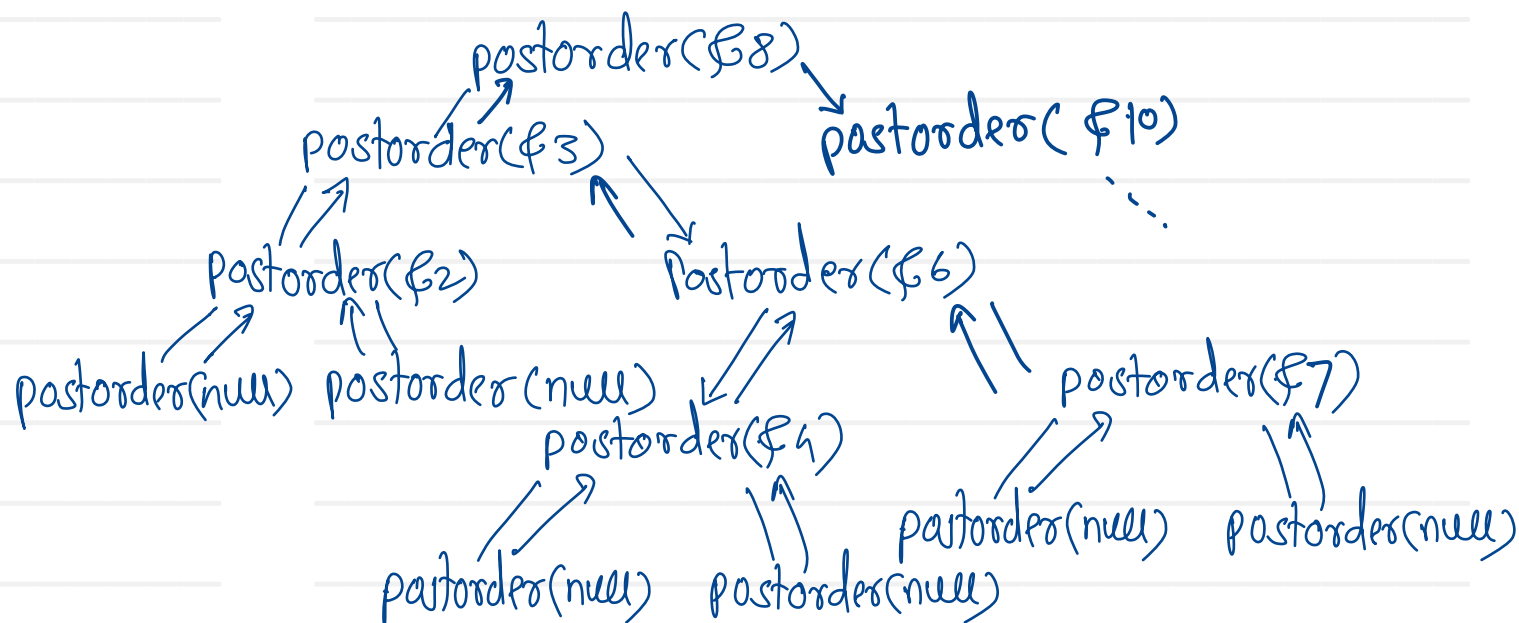
↑
root



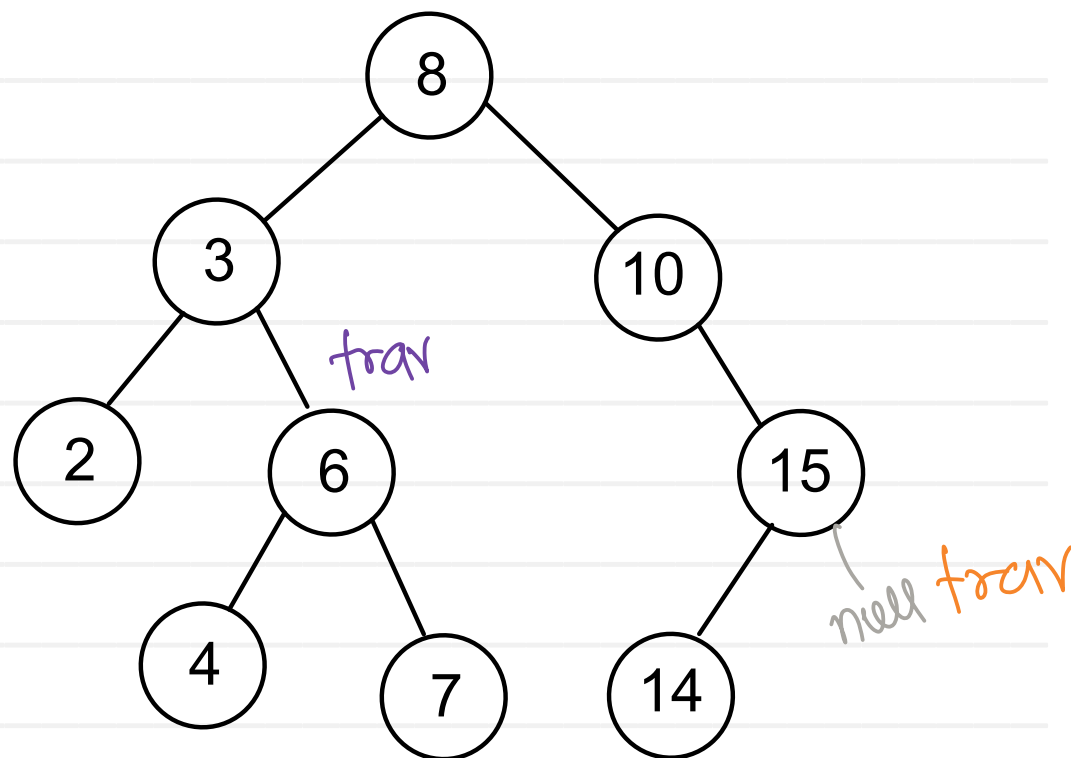


```

postorder(trav) {
    if(trav == null)
        return;
    postorder(trav.left);
    postorder(trav.right);
    sysout(trav.data);
}
  
```



Binary Search Tree - Binary Search



1. Start from root
2. If key is equal to current node data return current node
3. If key is less than current node data search key into left sub tree of current node
4. If key is greater than current node data search key into right sub tree of current node
5. Repeat step 2 to 4 till leaf node

Key = 6

trav

$8 <$

$3 >$

$6 =$

Key = 18

trav

$8 >$

$10 >$

$15 >$

null